

work

directly with package manager maintainers to enhance lock files with transitive dependency relationships, and dependency group metadata. For example, we could work with the Gradle maintainers to add a new version of their gradle.lockfile that includes parent dependencies. Our contributions would have the added benefit of improving the experience for our users by including the necessary tooling out of the box, overall improving the workflow for getting started with GitLab's dependency scanning feature.

Pros

- Does not require establishing new file requirements.
- Works out of the box in majority of cases. Package managers usually generate a lock file if one doesn't exist.

Cons

- Package managers tend to have large code bases that increase the onboarding time required.
- Lock files require domain expertise. For example, in pnpm's issue 7685 you can see the discussion of a very specific corner case that must be handled.
- Project maintainers have their own sets of concerns that may not align with our own. For example, they may prioritize stability and maintenance over new features.
- Older versions of package managers, or build tools, would not be compatible with new additions.

Rely on 3rd party CycloneDX generators

This approach moves the direction of composition analysis so that we interface only with user provided cyclonedx CI reports from 3rd party CycloneDX generators.

Pros

- No CI/CD component integration testing.
- No analyzer maintenance required.

Cons

- Tied to the GitLab release schedule, so we can't deploy new features, enhancements, and bug fixes mid milestone.
- There are a lot of third party analyzers that can generate a CycloneDX report. Supporting all of their custom metadata properties and component properties can be challenging.
- Requires time to get up to speed with third party SBOM generator code bases.
- Proposals to the generators may be rejected. If required, we could fork the project, but that comes with its own set of challenges.
- Dependency graphs may be incomplete like cyclondexpy in some circumstances.

Generate custom dependency graph exports with package manager plugins

In the cases where package managers expose a public API, we are able to write a plugin to generate the dependency graph in a format of our choosing. This has been used for gemnasium-maven dependency analysis.

Pros

- Choice of output format.
- Can re-use the bundled gemnasium-maven plugins.

Cons

- Not all package managers have support for third party plugins. For example, pnpm does not have documented plugin support.
- Plugins that do not use Ruby or Go require new language expertise, which lead to a smaller pool of plugin maintainers, and higher review load per maintainer.
- Additional overhead required for the maintenance, improvements, and deployments of plugin projects.

Appendix

- dependency graph export
- package manager
- lock file

SECTION: engineering/architecture/design-documents/deploy-workspaces-with-ci-runner/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

Configuring and deploying Workspaces in GitLab today depends on the customer having configured a Kubernetes cluster as well as a GitLab Kubernetes Agent and several other components. There is quite a lot of friction in the setup and even just running Kubernetes is likely a considerable burden for customers.

As well as the complexity of setting up Kubernetes to run Workspaces it may also not be very cost effective for some customers as the baseline workload of a Kubernetes cluster without any workspaces running is not insignificant.

This design document proposes that we allow customers to also deploy Workspaces using the existing CI Runners they have configured for their projects.

Proposal

This design document is related to <https://gitlab.com/gitlab-com/content-sites/handbook/-/mergerequests/10804> except this document focuses on deploying workspaces as opposed to how to get network access to the GitLab VS Code fork running inside the container.

The idea of this proposal is that we add an additional option to workspace creation to create the workspace using CI Runners. Behind the scenes this will skip all the Kubernetes agent related logic of Workspaces and create a CI Pipeline. The CI pipeline will use the same docker image specified in their Devfile and will inject the GitLab VS Code fork into that docker image in a similar way to how it is injected into our Kubernetes Workspaces.

This idea was demonstrated in this video demo which is composed of POC changes in the following merge requests:

1. <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/176479>
1. <https://gitlab.com/gitlab-org/gitlab-agent/-/mergerequests/2084>
1. <https://gitlab.com/gitlab-org/workspaces/gitlab-workspaces-tools/-/mergerequests/19>

Key benefits of CI Runners

It is clear that CI Runners and CI (in general) is quite different to the way workspaces work. So it is reasonable to ask why we want to try to make these things fit. Some of the main advantages I see in iterating along this path are:

1. CI Runners are a mature product that self-managed and GitLab.com customers are already using
1. CI Runners already know how to speak to multiple cloud providers (AWS, GCP, Azure) and even support creating K8s pods which could even replace our existing approach to provisioning workspaces in K8s. In addition CI Runners are based on a plugin system which allows users to implement their own cloud provider support
1. Expanding runners to support these more generic workloads will easily extend to Duo Workflow where we are struggling with this exact same problem of provisioning a workspace (see <https://gitlab.com/groups/gitlab-org/-/epics/16050>) and there are yet more proposals to better utilise Runners for more generic workloads (see <https://gitlab.com/gitlab-org/gitlab/-/issues/328489>)

Technical details

There is a POC in <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/176479> which shows roughly how we might support runner type workspaces as well as agent type workspaces.

Injecting the GitLab VS Code fork

The GitLab VS Code fork can be injected into the CI Job in a similar way to what we do for our existing Kubernetes functionality. Our `.gitlab-ci.yml` syntax already supports additional containers (called services) which have a shared volume. We could use this with our existing workspace tools images to inject the GitLab VS Code fork in almost an identical way. Then we can add commands to the script of the CI Job to run the GitLab VS Code fork.

Keeping things running (and declarative programming)

Many of the discussions around alternatives to Kubernetes for running these kinds of workloads focus on the benefits of the declarative API for Kubernetes. Specifically people like the idea that they can just declare all the workloads that they expect to be running and Kubernetes will take care of starting the new ones, shutting down ones that are no longer needed and also restarting processes as they crash.

When we use the CI Pipeline and Runner as the primary API for provisioning compute we won't be getting any of that "for free". We can rely on the CI Runner to shut down jobs after some time as well as a best effort to shut down jobs we cancel from the Rails side. But we'll still want some more robust "restart" mechanism for crashed workspaces. A simple option for this will be some kind of periodic process on GitLab Rails which is looping over all the workspaces we expect to be running and checking whether or not they are still running.

Due to network access this would likely require a kind of health check pushed by the workspace or runner. This may be able to make use of the existing runner heartbeat.

Those implementation details will be left out of this document, but we should assume that Rails can know whether or not the Workspace has reported it's status with some frequency.

Once we know if a workspace is no longer running (or healthy) and we believe it should be, then GitLab Rails will do 2 things:

1. Cancel the existing job for the workspace: notifying the runner to terminate it if it thinks it is still running
2. Create a new job for the workspace: and a runner will pick it up and create it again

All of this sounds much less elegant than the declarative APIs provided by Kubernetes but ultimately this is the kind of thing that is abstracted away by Kubernetes. Nothing prevents us from providing similarly "elegant" abstractions in the GitLab Rails codebase for hiding all this complexity.

Persistence

GitLab CI jobs already support persistence via a "cache". The cache storage in a CI Job usually involves zipping up a directory, uploading to object storage and then downloading and unzipping it the next time a CI job runs with the same cache key. We can likely re-use this mechanism initially to reduce the scope of

our first iteration. Using the CI cache will need to ensure that users do not accidentally get the same cached data as another user. This should be possible by using a unique cache key for each workspace.

The CI cache, however, is not likely to be a good long term solution due to cost of zip/unzip and durability. As such we will look to replace this quickly with a better solution. The most promising option here will be to create a volume per workspace and re-use that volume when a workspace is restarted. The volumes should only be destroyed when the workspace is removed and not when the workspace is stopped. Extending the CI Runner to create persistent volumes may align with similar goals for faster cache retrieval in CI jobs.

A related note here is that one of the key benefits of cloud development environments is that they are easy to recreate from scratch and this optimizes onboarding for new developers as well as provides consistent development environments across a team. While there may be a desire for developers to completely make a workspace their own, after they boot it up, it may also be an anti-pattern with regards to cloud development environments. Having long running workspaces has tradeoffs for keeping up with changes to the base image, contributing back workspace improvements to the team and accidentally creating a snowflake VM that other team members cannot recreate. Instead it may be preferable for GitLab to improve the tooling for developers to create persistent customizations for their workspace (e.g. separate git-managed config files that apply on startup). This is a side note due to the emphasis I've heard placed on persistence in my discussions and is otherwise not explored in this document.

Primarily we should aim to at least provide similar persistence in our CI based workspaces as compared to our existing Kubernetes based workspaces but we don't need to aim to improve on this as part of this architecture.

Devfile vs. Development Container vs. something else

Right now our workspace functionality uses the devfile format. Since this tool only really supports Kubernetes it may be a good time to move away from Devfile and try a different configuration file. We could consider moving to supporting Development Containers but that might lead to us needing to build a lot more functionality into GitLab.

For now we should attempt to start by just translating the minimum amount of details we need from the Devfile syntax to create a comparable workspace inside of CI. We should probably investigate the move to Development Containers or another format independently of this work.

If we find that extensive effort is needed to translate the Devfile to something that can run in a CI job then we might be forced to tackle this problem sooner.

Iteration plan

1. Introduce CI::Workload concept which allows creating a workload without defining a .gitlab-ci.yml

1. Alpha support for creating a workspace using Ci::Workload. There is no guarantee it stays alive at this point but it will be created.
1. Implement persisting and restoring the workspace code directory
1. Implement robust restarts of workspace if it crashes (or the CI Job times out)

Alternatives Considered

Use new service to manage the VMs

We could still re-use parts of the Runner stack including the fleeting API. This would at least reduce duplicated effort to integrate with these cloud providers and also provide a plugin systems for customers to implement their own cloud provider support. The downside might be similar to re-using all of CI Runners, because there still may be differences in how we manage VMs that make the code more difficult to work with.

Use our existing Google Cloud IAM integration

We have an integration with Google Cloud IAM. This integration provides a way for customers to connect their cloud accounts to GitLab and a mechanism inside GitLab for provisioning short-lived credentials. It could form the basis of an integration for provisioning VMs for workspaces but today it only provides authentication which would leave us to build everything else.

SECTION: engineering/architecture/design-documents/disaster_recovery/_index.md

{{< engineering/design-document-header >}}

This document is a work-in-progress and proposes architecture changes for the GitLab.com SaaS. The goal of these changes are to maintain GitLab.com service continuity in the case a regional or zonal outage.

For the current state see Disaster Recovery Policies for GitLab Backups.

- A zonal recovery is required when all resources are unavailable in one of the three availability zones in us-east1 or us-central1.
- A regional recovery is required when all resources become unavailable in one of the regions critical to operation of GitLab.com, either us-east1 or us-central1.

Services not included in the current DR strategy for FY24 and FY25

We have limited the scope of DR to services that support primary services (Web, API, Git, Pages, Sidekiq, CI, and Registry).

These services tie directly into our overall availability score (internal link) for GitLab.com.

For example, DR does not include the following:

- AI services including code suggestions
- Error tracking and other observability services like tracing
- CustomersDot, responsible for billing and new subscriptions
- Advanced Search

DR Implementation Targets

The FY24 targets were:

	Recovery Time Objective (RTO)	Recovery Point Objective (RPO)
Zonal	2 hours	1 hour
Regional	96 hours	2 hours

The FY25 targets before cell architecture are:

	Recovery Time Objective (RTO)	Recovery Point Objective (RPO)
Zonal	0 minutes	0 minutes
Regional	48 hours	0 minutes

Note: While the RPO values are targets, they cannot be met exactly due to the limitations of regional bucket replication and replication lag of Gitaly and PostgreSQL.

Current Recovery Time Objective (RTO) and Recovery Point Objective (RPO) for Zonal Recovery

We have not yet simulated a full zonal outage on GitLab.com.

The following are RTO/RPO estimates based on what we have been able to test using the disaster recovery runbook.

It is assumed that each service can be restored in parallel.

A parallel restore is the only way we are able to meet the FY24 RTO target of 2 hours for a zonal recovery.

Service	RTO	RPO
PostgreSQL	1.5 hr	<=5 min
Redis ^[1]	0	0
Gitaly	30 min	<=1 hr
CI	30 min	not applicable
Load balancing (HAProxy)	30 min	not applicable
Frontend services (Web, API, Git, Pages, Registry) ^[2]	15 min	0
Monitoring (Prometheus, Grafana, Alerting)	0	not applicable
Operations (Deployments, runbooks, operational tooling, Chef) ^[3]	30 min	4 hr
PackageCloud (distribution of packages for self-managed)	0	0

Current Recovery Time Objective (RTO) and Recovery Point Objective (RPO) for Regional Recovery

Regional recovery requires a complete rebuild of GitLab.com using backups that are stored in

multi-region buckets.

The recovery has not yet been validated end-to-end, so we don't know how long the RTO is for a regional failure.

Our target RTO for FY25 is to have a procedure to recover from a regional outage in under 48 hours.

The following are considerations for choosing multi-region buckets over dual-region buckets:

- We operate out of a single region so multi-region storage is only used for disaster recovery.
- Although Google recommends dual-region for disaster recovery, dual-region is not an available storage type for disk snapshots.
- To mitigate the bandwidth limitation of multi-region buckets, we spread Gitaly VMs infra across multiple projects.

Proposals for Regional and Zonal Recovery

- Regional
- Zonal

[^1]: Most of the Redis load is on the primary node, so losing replicas should not cause any service interruption

[^2]: We setup maximum replicas in our Kubernetes clusters servicing front-end traffic, this is done to avoid saturating downstream dependencies. For a zonal failure, a cluster reconfiguration is necessary to increase these maximums.

[^3]: There is a 4 hr RPO for Operations because Chef is a single point of failure in a single availability zone and our restore method uses disk snapshots, taken every 4 hours. While most of our Chef configuration is also stored in Git, some data (like node registrations) are only stored on the server.

SECTION: engineering/architecture/design-documents/disaster_recovery/regional.md

Improving the Recovery Time Objective (RTO) and Recovery Point Objective (RPO) for Regional Recovery

The following list the top challenges that limit our ability to drive RTO to 48 hours for a regional recovery.

1. We have a large amount of legacy infrastructure managed using Chef. This configuration has been difficult for us to manage and would require a large amount of manual copying and duplication to create new infrastructure in an alternate region.
1. Operational infrastructure is located in a single region, us-central1. For a regional failure in this region, it requires rebuilding the ops infrastructure with only local copies of runbooks and tooling scripts.
1. Observability is hosted in a single region.
1. The infrastructure (dev.gitlab.org) that builds Docker images and packages is located in a single region, and is a single point of failure.

1. There is no launch-pad that would allow us to get a head-start on a regional recovery. Our IaC (Infrastructure-as-Code) does not allow us to switch regions for provisioning.
1. We don't have confidence that Google can provide us with the capacity we need in a new region, specifically the large amount of SSD necessary to restore all of our customer Git data.
1. We use Global DNS for internal DNS making it difficult to use multiple instances with the same name across multiple regions, we also don't incorporate regions into DNS names for our internal endpoints (for example dashboards, logs, etc).
1. If we deploy replicas in another region to reduce RPO we are not yet sure of the latency or cloud spend impacts.
1. We have special/negotiated Quota increases for Compute, Network, and API with the Google Cloud Platform only for a single region, we have to match these quotas in a new region, and keep them in sync.
1. We have not standardized a way to divert traffic at the edge from 1 region to another.
1. In monitoring, and configuration we have places where we hardcode the region to us-east1.

Regional recovery work-streams

The first step of our regional recovery plan creates new infrastructure in the recovery region that involves a large number of manual steps.

To give us a head-start on recovery, we propose a "regional bulkhead" deployment in a new GCP region.

A "regional bulkhead" meets the following requirements:

1. A specific region is allocated.
1. Quotas are set and synced so that we can duplicate all of us-east1 in the new region.
1. Subnets are allocated or reserved in the same VPC for us-east1.
1. Some infrastructure is deployed where it makes sense to lower RTO, while keeping cloud-spend low.

The following are work-streams that can be done mostly in parallel.

The end-goal of the regional recovery is to have a bulkhead that has the basic scaffolding for deployment in the alternate region.

This bulkhead can be used as a launching pad for a full data restore from us-east1 to the alternate region.

Select an alternate region

We are going with us-central1. Discussion for this was done in <<https://gitlab.com/gitlab-com/gl-infra/production-engineering/-/issues/25094>>

- Dependencies: none
- Teams: Ops

The following are considerations that need to be made when selecting an alternate region for DR:

1. Ensure there is enough capacity to meet compute usage.
1. Network and network latency requirements, if any.
1. Feature parity between regions.

Deploy Kubernetes clusters supporting front-end services in a new region with deployments

- Dependencies: External front-end load balancing
- Teams: Ops, Foundations, Delivery

GitLab.com has Web, API, Git, Git HTTPs, Git SSH, Pages, and Registry as front-end services. All of these services are run in 4 Kubernetes clusters deployed in us-east1.

These services are either stateless or use multi-region storage buckets for data.

In the case of a failure in us-east1, we would need to rebuild these clusters in the alternate region and set them up for deployments.

Switch from Global to Zonal DNS

- Dependencies: None
- Teams: Gitaly

Gitaly VMs are single points of failure that are deployed in us-east1.

The internal DNS naming of the nodes have the following convention:

```
plaintext
gitaly-01-stor-gprd.c.gitlab-gitaly-gprd-ccb0.internal
^ name           ^ project
```

By switching to zonal DNS, we can change the internal DNS entries so they have the zone in the DNS name:

```
plaintext
gitaly-01-stor-gprd.c.us-east1-b.gitlab-gitaly-gprd-ccb0.internal
^ name           ^ zone   ^ project
```

Allowing us to keep the same name when recovering into a new region or zone.

```
plaintext
gitaly-01-stor-gprd.c.us-east1-b.gitlab-gitaly-gprd-ccb0.internal
gitaly-01-stor-gprd.c.us-east4-a.gitlab-gitaly-gprd-ccb0.internal
```

For fleets of VMs outside of Kubernetes, these names allow us to have the same node names in the recovery region.

Gitaly

- Dependencies: Switch from Global to Zonal DNS (optional, but desired)
- Teams: Gitaly, Ops, Foundations

Restoring the entire Gitaly fleet requires a large number of VMs deployed in the alternate region.

It also requires a lot of bandwidth because restore is based on disk snapshots.

To ensure a successful Gitaly restore, quotas need to be synced with us-east1 and there needs to be end-to-end validation.

PostgreSQL

- Dependencies: Improve Chef provisioning time by using preconfigured golden OS images (optional, but desired), local backups in the standby region (data disk snapshot and WAL archiving).
- Teams: Database Reliability, Ops

The configuration for Patroni provisioning only allows a single region per cluster.

There is networking infrastructure, Consul, and load balancers that need to be setup in the alternate region.

We may consider setting up a "cascaded cluster" for the databases to improve recovery time for replication.

Redis

- Dependencies: Improve Chef provisioning time by using preconfigured golden OS images (optional, but desired)
- Teams: Ops

To provision Redis subnets need to be allocated in the alternate region with and end-to-end validation of the new deployments.

External front-end load balancing

- Dependencies: HAProxy replacement, mostly likely GKE Gateway and Istio
- Teams: Ops, Foundations

External front-end load balancing is necessary to validate the deployment in the alternate region.

This requires both external and internal LBs for all front-end-services.

Monitoring

- Dependencies: Eliminate X% Chef dependencies in Infra by moving infra away from Chef (migrate Prometheus infra to Kubernetes)
- Teams: Observability, Ops, Foundations

Setup an alternate ops Kubernetes cluster in a different region that is scaled down to zero replicas.

Runners

Dependencies: Improve Chef provisioning time by using preconfigured golden OS images (optional, but desired)

Teams: Scalability:Practices, Ops, Foundations

Ensure quotas are set and align with us-east1 in the alternate region for both runner managers

and ephemeral VMs.

Setup and validate networking configuration with peering configuration.

Ops and Packaging

- Dependencies: Create an HA Chef server configuration to avoid an outage for a single zone failure
- Teams: Scalability:Practices, Ops, Foundations, Distribution

All image creation and packaging is done on a single VM, our operation tooling is also on a single VM.

Both of these are single points of failures that have data stored locally.

In the case of a regional outage, we would need to rebuild them from snapshot and lose about 4 hours of data.

The following are options to mitigate this risk:

- Move our packaging jobs to ops.gitlab.net so we eliminate dev.gitlab.org as a single point of failure.
- Use the Geo feature for ops.gitlab.net.

Regional Recovery Gameday

- Dependencies: Recovery improvements
- Teams: Ops

Following the improvements for regional recovery, a Gameday needs to be executed for end-to-end testing of the procedure.

Once validated, it can be added to our existing disaster recovery runbook.

SECTION: engineering/architecture/design-documents/disaster_recovery/zonal.md

Improving the Recovery Time Objective (RTO) and Recovery Point Objective (RPO) for Zonal Recovery

The following represents our current DR challenges and are candidates for problems that we should address in this architecture blueprint.

1. Postgres replicas run close to capacity and are scaled manually. New instances must go through Terraform CI pipelines and Chef configuration. Over-provisioning to absorb a zone failure would add significant cloud-spend (see proposal section at the end of the document for details).
1. HAProxy (load balancing) is scaled manually and must go through Terraform CI pipelines and Chef configuration.
1. CI runner managers are present in 2 availability zones and scaled close to capacity. New instances must go through Terraform CI pipelines and Chef configuration.
1. In a zone there are saturation limits, like the number of replicas that need

to be manually adjusted if load is shifted away from a failed availability zone.

1. Gitaly RPO is limited by the frequency of disk snapshots, RTO is limited by the time it takes to provision and configure through Terraform CI pipelines and Chef configuration.
1. Monitoring infrastructure that collects metrics from Chef managed VMs is redundant across 2 availability zones and scaled manually. New instances must go through Terraform CI pipelines and Chef configuration.
1. The Chef server which is responsible for all configuration of Chef managed VMs is a single point of failure located in us-central1. It has a local Postgres database and files on local disk.
1. The infrastructure (dev.gitlab.org) that builds Docker images and packages is located in a single region, and is a single point of failure.

Zonal recovery work-streams

Improvements around zonal recovery revolve around improving the time it takes to provision for fleets that do not automatically scale.

There is already work in-progress to completely eliminate statically allocated VMs like HAProxy.

Additionally efforts can be made to shorten launch and configuration times for fleets that are not able to automatically scale like Gitaly, PostgreSQL and Redis.

Over-provision to absorb a single zone failure

- Dependencies: None
- Teams: Ops, Scalability:Practices, Database Reliability

All of our Chef managed VM fleets run close to capacity and require manual scaling and provisioning using Terraform/Chef.

In the case of a zonal outage, it is necessary to provision more servers through Terraform which adds to our recovery time objective.

One way to avoid this is to over-provision so we have a full zone's worth of extra capacity.

1. Patroni Main (n2-highmem-128 6.5k/month): 3 additional nodes for +20k/month
1. Patroni CI (n2-highmem-96 5k/month): 3 additional nodes for +15k/month
1. HAProxy (t2d-standard-8 285/month): 20 additional nodes for +5k/month
1. CI Runner managers (c2-standard-30 1.3k/month) 60 additional nodes for +78k/month

The Kubernetes horizontal auto-scaler (HPA) has a maximum number of pods configured on front-end services.

It is configured to protect downstream dependencies like the database from saturation due to scaling events.

If we allow a zone to scale up rapidly, these limits need to be adjusted or re-evaluated in the context of disaster recovery.

Remove HAProxy as a load balancing layer

- Dependencies: None
- Teams: Foundations

HAProxy is a fleet of Chef managed VMs that are statically allocated across 3 AZs in us-east1. In the case of a zonal outage we would need to rapidly scale this fleet, adding to our RTO.

In FY24Q4 the Foundations team started working on a proof-of-concept to use Istio in non-prod environments.

We anticipate in FY25 to have a replacement for HAProxy using Istio and GKE Gateway.

Completing this work reduces the impact to our LoadBalancing layer for zonal outages, as it eliminates the need to manually scale the HAProxy fleet.

Additionally, we spend around 17k/month on HAProxy nodes, so there may be a cloud-spend reduction if we are able to reduce this footprint.

Create an HA Chef server configuration to avoid an outage for a single zone failure

- Dependencies: None
- Teams: Ops

Chef is responsible for configuring VMs that have workloads outside of Kubernetes.

It is a single point of failure that resides in us-central1-b.

Data is persisted locally on disk, and we have not yet investigated moving it to a highly available setup.

In the case of a zonal outage of us-central1-b the server would need to be rebuilt from snapshot, losing up to 4 hours of data.

Create an HA Packaging server configuration to avoid an outage for a single zone failure

- Dependencies: None
- Teams: Ops

In the case of a zonal outage of us-east1-c the server (like dev.gitlab.org) would need to be rebuilt from snapshot, losing up to 4 hours of data.

The additional challenge of this host is that it is a GitLab-CE instance so we would be limited in features.

The best approach here would likely be to move packaging CI pipelines to ops.gitlab.net.

Improve Chef provisioning time by using preconfigured golden OS images

- Dependencies: None
- Teams: Ops

For the Gitaly fleet upgrade in 2022 a scheduled CI pipeline was created to build a golden OS images.

We can revive this work and start generating images for Gitaly and other VMs to shorten configuration time.

We estimate that using an image can reduce our recovery time by about 15 minutes to improve RTO for zonal failures.

Eliminate X% Chef dependencies in Infra by moving infra away from Chef

- Dependencies: None

- Teams: Ops, Observability

Gitaly, Postgres, CI runner managers, HAProxy, Bastion, CustomersDot, Deploy, DB Lab, Prometheus, Redis, SD Exporter, and Console servers are managed by Chef.

To help improve the speed of recoveries, we can move this infrastructure into Kubernetes or Ansible for configuration management.

Write-ahead-log for Gitaly snapshot restores

- Dependencies: None

- Teams: Gitaly

There is work planned in FY25Q1 that adds a transaction log for Gitaly to reduce RPO.

SECTION: engineering/architecture/design-documents/duo_workflow/_index.md

{{< engineering/design-document-header >}}

Execution Environment

Executive Summary

The functionality to support GitLab Duo Workflow needs to be able to execute arbitrary code which effectively means "untrusted" code. This means that they cannot just run like any other service we deploy and specifically they cannot just run inside the Workflow service or AI Gateway.

In order to address this issue, the Workflow functionality will be comprised of 2 separate components:

1. The Workflow service, which is a Python service we run in our infrastructure. The Workflow service is built on top of LangGraph.
1. The Workflow executor, which is a Go binary that communicates via a long running gRPC connection to the Workflow service and executes the arbitrary commands. It will be possible for users to run this locally or in CI pipelines.

In our first release we will support 2 execution modes:

1. Local executor: which will run commands and edit files locally in a sandboxed Docker container on the developer machine. They will be able to see the files being edited live and it will be interactive.
1. CI executor: All non-local use-cases of Workflow (for example: issue/epic based workflows) will be triggered by the GitLab UI and will create a CI Pipeline to run the Workflow executor.

Our architecture will also support mixed deployments for self-managed such that some features of Workflow will be available using a cloud-hosted AI

Gateway.

Detailed plan

We plan on building this feature set with 3 independent components that can be run in multiple runtimes:

1. The Workflow Web UI. This will be the web UI built into GitLab that manages the creation and interaction of all workflows. There may be many interaction points in the GitLab application but there should be a central workflow UI with reusable components (e.g. Vue components) that could be embedded into our editor extensions.
1. The Workflow service. This is a Python-based service we deploy with a gRPC API. The only interface to this is the gRPC interface, which is called from the Workflow executor. Internally, this will use LangGraph to execute the workflows. For reasons why LangGraph was chosen, see this work item. The Workflow service will not have any persisted state but the state of running workflows will be kept in memory and periodically checkpointed in GitLab. The Workflow service is built in its own codebase and will have its own deployment but the codebase may be merged with the AI Gateway codebase in the future.
1. The Workflow executor.
This is being written in Go for easy installation in development containers. This component will run in CI jobs or on a user's local workstation. In the local workstation it will run sandboxed in a Docker container with the working directory optionally mounted by the user for a live pairing experience. It will only be responsible for opening a gRPC connection to the Workflow service and executing the commands it is told to.

The following are important constraints of the architecture:

1. All state management will be inside GitLab.
1. Periodically, the Workflow service should checkpoint its state in GitLab.
1. The Workflow service in-memory state can be dropped/lost at any time so checkpointing will be the only guaranteed point that can be returned to.
1. If a local Workflow executor drops the connection, the Workflow service will checkpoint and shutdown the state as soon as it runs into something where it is waiting on the executor.
1. To avoid multiple Workflow service instances running on the same workflow, the Workflow service must always acquire a lock with GitLab before it starts running. When it suspends, it will release the lock and similarly there will be a timeout state if it has not checkpointed in the last 60 seconds. GitLab will not accept checkpoints from a timed out run of the Workflow service.
1. Each time a Workflow service resumes a workflow, it gets a new ID. This ID is sent when checkpointing so that GitLab can drop/ignore zombie services running the workflow and inform the zombie service to shutdown.
1. Code is checkpointed by the executor pushing hidden Git refs to the GitLab instance. This will be happening on the same frequency as other checkpoints.

1. For local execution, the Workflow executor initiates a workflow directly by calling the Workflow service.
1. When a workflow is triggered from the UI, it doesn't require a Workflow executor. GitLab can call the Workflow service directly.
1. All API calls from the Workflow service to GitLab that access private data or update data will be authenticated on behalf of the user that created the workflow. The Workflow service should not need privileged access to GitLab.

CI pipelines have been chosen as the hosted runtime option for the Workflow executor because it is the only infrastructure we have available today to run untrusted customer workloads with stability, support, security, abuse prevention and a billing model. In the short term for early customers we may rely on the existing compute minutes for CI pipelines but in the long run we may want to deploy dedicated runners and introduce a billing model specific for Workflow.

For many development use cases we expect developers may prefer to run the Workflow executor locally as it can operate on a locally mounted directory and allow the user to more easily watch changes as they happen.

GitLab.com architecture

1. Initially we focus on running locally and in CI pipelines with all inputs as environment variables.
1. State stored in GitLab so it can be accessed from the web UI and through IDE extensions.

With Local (IDE) execution

```

mermaid
sequenceDiagram
    participant user as User
    participant ide as IDE
    participant executor as Workflow executor
    participant gitlabrails as GitLab Rails
    box AI-gateway service
        participant duoworkflowservice as Workflow service
        participant aigateway as AI Gateway
    end
    participant llmprovider as LLM Provider

    user->>ide: trigger workflow from IDE
    ide->>gitlabrails: Create the workflow
    gitlabrails->>gitlabrails: Create JWT for Workflow service
    gitlabrails->>gitlabrails: Create aiworkflow scoped OAuth token
    gitlabrails->>gitlabrails: Create the workflow
    gitlabrails->>ide: Return the workflow details and JWT and OAuth tokens
    ide->>executor: start executor with workflow details and JWT and OAuth token
    executor->>+duoworkflowservice: Solve this issue (open grpc connection auth'd with AI

```

Gateway JWT)

```
duoworkflowservice->>llmprovider: Ask LLM what to do
llmprovider->>duoworkflowservice: Run rails new mynewapp
duoworkflowservice->>executor: execute rails new mynewapp
executor->>duoworkflowservice: result rails new mynewapp
duoworkflowservice->>gitlabrails: Save checkpoint
duoworkflowservice->>llmprovider: What's next?
llmprovider->>duoworkflowservice: You're finished
duoworkflowservice->>gitlabrails: Save checkpoint and mark completed
duoworkflowservice->>gitlabrails: Revoke aiworkflow scoped OAuth token
deactivate duoworkflowservice
gitlabrails->>user: Workflow done!
```

With Remote (CI pipeline) execution

mermaid

sequenceDiagram

autonumber

participant user as User

participant gitlabrails as GitLab Rails

box CI-Runner LightYellow

participant executor as Workflow executor

end

box AI-gateway service LightBlue

participant duoworkflowservice as Workflow service

participant aigateway as AI Gateway

end

participant llmprovider as LLM Provider

note over user,gitlabrails: User is logged in via web

user->>gitlabrails: trigger workflow from Web UI

gitlabrails->>gitlabrails: Create JWT for Workflow service

gitlabrails->>gitlabrails: Create aiworkflow scoped composite identity OAuth token

gitlabrails->>gitlabrails: Create the workflow

gitlabrails->>executor: start executor in CI pipeline with workflow details and JWT and composite identity OAuth token

note over executor,duoworkflowservice: AI Gateway JWT

executor->>+duoworkflowservice: Solve this issue (open gRPC connection)

note over duoworkflowservice,llmprovider: API Key (from env)

duoworkflowservice->>llmprovider: Ask LLM what to do

llmprovider-->>duoworkflowservice: Run rails new mynewapp

note over duoworkflowservice,executor: Authenticated gRPC

duoworkflowservice->>executor: execute rails new mynewapp

executor-->>duoworkflowservice: command result

note over duoworkflowservice,gitlabrails: Composite OAuth token
duoworkflowservice->>gitlabrails: Save checkpoint

note over duoworkflowservice,Ilmprovider: API Key (from env)
duoworkflowservice->>Ilmprovider: What's next?
Ilmprovider-->>duoworkflowservice: You're finished

note over duoworkflowservice,gitlabrails: Composite OAuth token
duoworkflowservice->>gitlabrails: Save checkpoint & mark completed
duoworkflowservice->>gitlabrails: Revoke Composite OAuth token
deactivate duoworkflowservice

note over gitlabrails,user: No Auth Required
gitlabrails->>user: Workflow done!

Self-managed architecture

With local Workflow service

When customers are running the Workflow service locally the architecture will be very similar to GitLab.com. This will also allow them to use whatever customer models they configure in their Workflow service.

With cloud Workflow service

In order to allow self-managed customers to trial and rapidly adopt Duo Workflow without running all Workflow service components, this architecture will supported a mixed deployment mode. In this case, we assume that the cloud AI Gateway will not have access to the customers GitLab instance but we can make use of the local executor (on the user's machine or in a CI runner) to proxy all interactions with GitLab.

Running without the executor

As described above there are 2 reasons we need the Workflow executor:

1. We need a sandboxed environment to safely execute arbitrary commands generated by the AI
1. We need a path to proxy requests to self-managed GitLab instances that may not be visible to our cloud Workflow service

But we will have a subset of use cases for Workflow where these conditions will not apply. Specifically we expect to have "non-code" workflows (e.g. review this merge request) where we just need to interact between LLM and GitLab APIs. And if the customer is using GitLab.com or a self-hosted AI

Gateway that has access to their GitLab instance then we can safely run all of this inside the Workflow service making API calls to the GitLab instance.

This architecture unlocks a considerable advantage as we expect the cost of running workloads in CI pipelines is quite high (and indeed wasteful) and the effort to get a local executor running is also quite high and it may limit the value for simple non-code workflows if these were a requirement. Additionally we get quite significant scaling advantages if we never have to create a CI pipeline to do some of these non-code workflows as starting up a pipeline and keeping it running for the duration of a workflow is a large overhead.

We may choose to support this architecture later but it will depend on the following design decisions:

1. The workflow should know how to call GitLab directly for several API calls and especially checkpointing.
1. Proxying to GitLab via the executor should be designed as an optional proxy where the Workflow service constructs the full HTTP request, but under certain configurations, will choose to pass the HTTP request to the executor instead of calling GitLab directly.
1. The Workflow executor is optional for workflows. The workflow should only suspend when the workflow depends on an executor and one is not present. When the workflow suspends it should know to checkpoint in GitLab before shutting down.
1. The Workflow service will need to acquire some kind of "lease" from the GitLab instance while it is running to prevent the possibility of 2 instances of the same workflow running concurrently or to prevent a crashed workflow from being indefinitely locked up and cannot be resumed. Leases should timeout and API calls to checkpoint (or perform other operations) may be rejected if they come from an instance of the workflow with an expired lease.

Data flow

The below diagram shows what happens when the user is triggering workflows from their IDE using a local executor. The architecture will be similar when triggering from the GitLab UI using CI pipelines except that GitLab will start a CI pipeline to create run the Workflow executor and create the workflow.

```
mermaid
sequenceDiagram
    participant user as User
    participant ide as IDE
    participant executor as Workflow executor
    participant gitlabrails as GitLab Rails
    participant duoworkflowservice as Workflow service
    participant llmprovider as LLM Provider
    user->>ide: trigger workflow from IDE
```

```

ide->>executor: start executor
executor->>+duoworkflowservice: Solve this issue
duoworkflowservice->>gitlabrails: Create the workflow
duoworkflowservice->>llmprovider: Ask LLM what to do
llmprovider->>duoworkflowservice: Need the file list
duoworkflowservice->>executor: execute ls
duoworkflowservice->>gitlabrails: Save checkpoint
executor->>duoworkflowservice: result ls
duoworkflowservice->>llmprovider: What's next?
llmprovider->>duoworkflowservice: Here's a patch
duoworkflowservice->>executor: execute git apply
duoworkflowservice->>gitlabrails: Save checkpoint
duoworkflowservice->>executor: execute poetry run pytest
duoworkflowservice->>gitlabrails: Save checkpoint
executor->>duoworkflowservice: result poetry run pytest
duoworkflowservice->>llmprovider: fix the tests
llmprovider->>duoworkflowservice: Here's a patch
duoworkflowservice->>executor: execute git apply
duoworkflowservice->>gitlabrails: Save checkpoint
duoworkflowservice->>executor: execute poetry run pytest
executor->>duoworkflowservice: result poetry run pytest
duoworkflowservice->>executor: Next step?
executor->>gitlabrails: Check in & Next step?
gitlabrails->>executor: Last step!
executor->>duoworkflowservice: Done!
deactivate duoworkflowservice
gitlabrails->>user: Workflow done!

```

CI pipeline architecture

We don't want users to have to configure a specific `.gitlab-ci.yml` in order to support Workflow. In order to avoid this we'll use the same approach as that used by DAST site validations which dynamically constructs a pipeline configuration in GitLab and triggers the pipeline without using any `.gitlab-ci.yml`.

CI pipelines also must be run inside a project. There will be some usecases of Workflow where there is no appropriate project in which to run the pipeline (e.g. bootstrapping a new project). For these workflows we will:

1. Initially require the user to have a default Workflow project created. It can just be any empty project and we'll automatically run the pipeline there.
1. If this proves to be too much setup we'll automate the creation of a default Workflow project for you.
1. If the UX is poor over time we might abstract the user away from the existence of the Project altogether and make this an implementation detail. This will be considered a last resort because it could be quite a wide impacting change to GitLab as projects are a central part of GitLab.

Considerations for CI Runners and Infrastructure

1. Our Workflow rollout may involve substantial increases to our CI runner usage.
1. Workflow will likely involve running long running CI pipelines that use very little CPU. Mostly what they will be doing is communicating back and forth with the LLMs and users in a long running gRPC connection.
1. Users will expect very low latency for CI runner startup.
 1. We should determine if there are ways to have preloaded VMs with our Docker images running ready to start a pipeline when it a workflow is triggered.
1. We likely want a set of CI runners that are just for Workflow. This may mean enabling the runners to a subset of customers or just using appropriate job labeling/runner matching to only use these runners for Workflow.
1. It might be possible to roll out some Workflow features on our existing runner fleets but we believe there will be enough benefits to invest in segregating these runners.

State checkpointing

The Workflow state will be persisted in GitLab-Rails as the Workflow service works. There are 2 components to state:

1. The State object being managed by Langgraph. This includes all prompt history between user and agents and any other metadata created by the LangGraph graph.
1. The working directory where the agent is writing code.
1. We will have data retention limits on all state. We will use PostgreSQL partitioning to drop old workflow data after some time and we will also drop old Git refs after some time.

We will be persisting the LangGraph state object using APIs in GitLab to persist this state to PostgreSQL as it goes. The API will use similar LangGraph conventions to identify all checkpoints with a thread as implemented in the POC <<https://gitlab.com/gitlab-org/gitlab/-/mergerequests/153551>>.

For the current working directory which contains the code the agent has written so far we will store this by pushing hidden Git refs to GitLab for the checkpoint. Each checkpoint will have an associated ref and a checkpoint naming convention (or something stored in PostgreSQL) will allow us to identify the appropriate Git ref for the state checkpoint.

Storing in Git has the advantage that we don't need to build any new API for storing artifacts and it's very easy for the user to access the code by just checking out that SHA. It also has huge storage savings where a workflow is working on an existing large project. Ultimately we expect code changes end up being pushed to Git anyway so this is the simplest solution.

Some workflows do not have an existing project (e.g. bootstrapping a project). Even those workflows will need to be triggered from some project (as

explained in the section about CI pipelines). As such we can use the workflow project as a temporary repository to store the snapshots of code generated by the workflow.

Consideration should also be made to cleanup Git refs over time after some workflow expiration period.

Authentication

GitLab Duo Workflow requires several authentication flows.

In this section, each connection that requires authentication is listed and the authentication mechanism is discussed.

Local Workflow executor -> Workflow service

When a workflow starts, the Workflow executor must connect to the Workflow service.

To authenticate this connection:

1. The IDE will use the OAuth token of Personal Access Token (PAT) that the user generated while setting up the GitLab editor extension.
1. The IDE uses that token to authenticate a request to a GitLab Rails API endpoint.
1. When the GitLab Rails API receives this request, it loads its instance-scoped JWT (synced daily from CustomersDot) and contacts the AI gateway to swap this instance token for the above-mentioned user-scoped token (also cryptographically signed).
1. GitLab Rails returns the user-scoped JWT to the IDE.
1. The IDE passes on this JWT to the local Workflow executor component.
1. The Workflow executor uses this JWT to authenticate the Workflow service gRPC connection.

This flow mimics the token flow that allows IDEs to connect direct to the AI Gateway.

CI Workflow executor -> Workflow service

When a workflow is executed by a CI runner, the Workflow executor must connect to the Workflow service.

A CI pipeline is created by GitLab, so there is no need to query a GitLab Rails API endpoint to obtain a short-lived user- and system-scoped JWT. Instead, in the process of creating the CI pipeline, GitLab Rails will:

1. Generate the user-scoped JWT.
1. Inject the JWT as an environment variable (for example: DUOWORKFLOWTOKEN)

in the CI pipeline.

1. The Workflow executor running inside the CI job uses this environment variable value to authenticate the Workflow service gRPC connection.

Workflow service -> GitLab Rails API

Reasons that the Workflow service must be able to authenticate requests to the GitLab Rails API:

1. The Workflow service will need to periodically make requests to GitLab Rails to sync workflow state. This means that the Workflow service must be able to authenticate these requests.
1. Workflow may need to make other GitLab Rails API queries to gather context. For example, a workflow for "solve issue with code" would require an API request to retrieve the issue content.
1. The end state of a workflow may take the form of a generated artifact (for example, Git commit or pull request) on the GitLab platform. To generate this artifact, the Workflow service must be able to make API requests to GitLab Rails.

Requirements for the token used to authenticate requests from the Workflow service to the GitLab Rails API:

1. Any artifacts created by Workflow must be auditable in order to maintain transparency about AI-generated activities on the GitLab platform.
1. The token's access level must match the access level of the user who initiated the Workflow to ensure that there is no privilege escalation.
1. We must have the ability to block read/write for all resources that belong to instances/projects/groups with `duofeaturesenabled` set to false.
1. Token must be valid for as long as it takes an agent to execute or be refreshable by the Workflow service. Workflow execution may take several hours.

The JWT that the Workflow executor uses to authenticate to the Workflow service could potentially be adapted to also work for this use-case but has some problems:

1. Need to update GitLab Rails to accept this type of token for API authentication.
1. JWTs are not revocable; what if we need to cut off an agent's access?
1. Need to build token rotation. How would the Workflow service authenticate an API request to generate a new token if the old JWT is already expired?

For these reasons, OAuth is a better protocol for this use-case. OAuth tokens:

1. Are only valid for 2 hours.
1. Can be revoked.
1. Have a built-in refresh flow.
1. Are an established authentication pattern for federating access between services.

Workflow service OAuth v1

To use OAuth, we will:

1. Create a new token scope called aiworkflows (related issue).
1. When the IDE requests the Workflow service User JWT from GitLab Rails, we will also generate and return an OAuth token with the aiworkflows scope. This OAuth token belongs to the user.
1. The Workflow executor will send that OAuth token, along with the baseurl of the GitLab Rails instance, as metadata when the Workflow service when the gRPC connection is opened.
1. The Workflow service will use the OAuth token for any GitLab Rails API Requests to read or write data for a Workflow.

Workflow service OAuth v2

As of October 18, 2024, the OAuth v1 flow has been implemented for Workflow.

The next iteration of Workflow OAuth (v2) will also use an OAuth token to authenticate requests to the GitLab API. But, instead of using a regular OAuth token, we will use a composite OAuth token. Composite tokens is a new concept that will require adding dynamic scopes to Doorkeeper, the library we use for OAuth.

The composite OAuth token will belong to a service account but will be tied to a human user. As a result, the output of Workflow will be attributed to a machine user but the access of the token will be the intersection of what the machine user's permissions and what the human user's permissions allow.

mermaid

graph TB

```
A[Action Request] --> B{Human User<br/>Has Permission?}
B -->|No| C[["Permission Denied"]]
B -->|Yes| D{Machine User<br/>Has Permission?}
D -->|No| E[["Permission Denied"]]
D -->|Yes| F[["Action Permitted"]]
```

To accomplish this, we will:

- Create a service account for the Workflow AI agent with its own distinct identity.
- For all GitLab instances (including GitLab.com): there will be one service account per instance.
- Generate a new GitLab OAuth application that accepts both the aiworkflows and user: scopes (latter scope is a "dynamic scope," which is what makes composite identity tokens possible).
- Generate an OAuth access token for the new OAuth application and service account user.
- In this scenario, the OAuth client and server are both GitLab. The request to authenticate comes in from either the IDE or GitLab. The IDE already has a

token for the user and GitLab exchanges that token for a service account token.

- Because GitLab is both the client and the server, there is no OAuth consent screen shown to the user during this flow. A group owner or instance admin will enable Workflow by setting up a Workflow service account. This is effectively the same thing as clicking "Authorize" on the consent screen which would authorize the 3rd-party app to use the service account.
- The OAuth access token will have the aiworkflows scope to narrow down the access permissions of the AI agent.
- The OAuth access token we will create for the AI Agent will have a human user scope (user:123 using the user id).

The authentication sequence for OAuth v2 identical to OAuth v1, the only difference is that the generated OAuth token is a composite token rather than a regular user OAuth token:

mermaid
sequenceDiagram
 participant IDE
 participant LSP
 participant DWE as Workflow executor (local)
 participant DWS as Workflow service
 participant GR as GitLab Rails (.com or SM)
 participant LLMs

IDE->>GR: API Request to return Workflow service User JWT (UJWT) and Service Account OAuth composite token with aiworkflows scope and dynamic user scope. Authenticated with: PAT or OAuth token used to authenticate GitLab Editor Extension.

GR->>DWS: API request to generate UJWT. Authenticated with: Instance JWT (IJWT)

IDE->>LSP: pass UJWT, OAuth token, and other metadata. Authenticated with: n/a, installed locally,

LSP->>DWE: pass UJWT, OAuth token, and other metadata. Authenticated with: n/a, installed locally.

DWE->>DWS: grpc connection. Pass OAuth token and other metadata to be used for API requests to GitLab Rails. Authenticated with: UJWT in header.

DWS->>LLMs: Authenticated with: API keys from environment

DWS->>GR: API requests to create the workflow, save checkpoints, and perform any other API requests required by the Workflow. Authenticate with: Service Account OAuth composite token with aiworkflows scope and dynamic user scope.

For more details, see Issue 480577.

Workflow service OAuth v3

The aiworkflows scope was added to ensure narrow token abilities.

There are 2 primary problems with the aiworkflows static scope approach:

- Not every Workflow will need access to the same API endpoints. By using the

same scope for every Workflow, we are providing more access than is necessary. This violates the principle of least privilege.

- Each GitLab API endpoint must be manually allow-listed for this scope.

Example.

This means that code changes are required to provide new functionality to Workflow, which is time intensive. This also means that new Workflow functionality may not be available to older GitLab versions where the scope has not yet been allow-listed for the necessary endpoints.

The solution to both of these problems is to provide dynamic scopes to OAuth access tokens. This means that the scope itself will determine which endpoints the token can access rather than relying on workflows or any other static token scope.

Advanced token scopes are being added to personal access tokens, secure job tokens, and are what will make routable tokens possible.

Dynamic token scopes are also being added to OAuth tokens in order to support composite identity (OAuth v2, described above). The plan for enabling targeted API access using dynamic token scopes is still in progress. The discussion on this topic is in Issue 468370.

Options we've considered and pros/cons

Delegate only unsafe execution to local/CI pipelines

This was the option we chose. It attempts to keep as much of the functionality as possible in services we run while delegating the unsafe execution to the Workflow executor which can run locally or in CI pipelines.

Pros:

1. Running the infrastructure ourselves gives us more control over the versions being rolled out.
1. There is less dependencies the user needs to install for local usage.
1. It offers a rapid onboarding experience for self-managed customers to try Workflow without deploying any new GitLab components.

Cons

1. We need to deploy and maintain new infrastructure which has different scaling characteristics to other services we run due to long running execution.

Run it locally

Pros:

1. This keeps developers in their local environment where most of them work.
1. Compute is absorbed by the local developer so they don't have to worry about

being billed per minute.

1. Low latency for user interaction especially where the user needs to review/edit code while the agent is working.

Cons:

1. There are more risks running it locally unless you have an isolated development environment as commands have full access to your computer. This can be mitigated by UX that limits what commands the agent can run without user confirmation.
1. This approach will require some local developer setup and may not be suited to tasks that users are expecting to kick off from the web UI (e.g. issue/epic planning).

CI pipelines (on CI runners)

See <<https://gitlab.com/gitlab-org/gitlab/-/issues/457959>> for a POC and investigation.

Pros:

1. CI pipelines are the only pre-configured infrastructure we have that can run untrusted workflows.
1. We have an established billing model for CI minutes.

Cons:

1. CI pipelines are slow to start up and this might mean that iteration and incremental AI development might be slow if the pipelines need to be restarted while timing out waiting for user input.
1. CI minutes will need to be consumed while the agent is awaiting for user input. This will likely require a timeout mechanism and as such if the user returns we'll need to restart a new pipeline when they give input.
1. CI pipelines run in a difficult to access environment (ie. you cannot SSH it or introspect it live) and as such it may make it difficult for users to interact with code that is being built out live in front of them without.
1. CI pipelines require there to be some project to run in. This is not likely something we can overcome but we may be able to simplify the setup process by automatically creating you a "workflow project" for your workflow pipelines to run in.
1. When we implement non-code workflows (e.g. reviewing MRs) there is no need for an isolated compute environment but we'll still be forcing customers to use compute minutes. We've seen this is not a good experience in other cases like X-Ray reports.

GitLab workspaces (remote development)

See <<https://gitlab.com/gitlab-org/gitlab/-/issues/458339>> for a POC and investigation.

Pros:

1. This has the fastest iteration cycle as the agent is working locally in your development environment and can interact with you and you can even see and edit the same files live as

them.

1. Customers can run it on their own infrastructure and this gives them control over efficient resource usage.

Cons:

1. Today we only support customers bringing their own infrastructure (K8s cluster) and this means that the barrier to getting started is to bring your own K8s cluster and this is a fairly significant effort.

1. If we wanted to build out infrastructure on GitLab.com to save customers having to bring their own K8s cluster this would be a fairly large effort from a security and infrastructure perspective. It's possible but to deal with all the complexities of security, abuse and billing would require many teams involvement in both initial development and sustained maintenance.

Security

Threat modeling

See <<https://gitlab.com/gitlab-com/gl-security/product-security/appsec/threat-models/-/issues/46>>.

Security considerations for local execution

Local execution presents the highest value opportunity for developers but also comes with the greatest risk that a bug or mistake from an LLM could lead to causing significant harm to a user's local development environment or compromise confidential information.

Some examples of risks:

1. An AI that can make honest but significant mistakes.

1. An AI that might sometimes be adversarial.

1. The AI gateway serving the LLM responses may be compromised which would then allow shell access to all users of this tool.

Sandboxing Workflow executor

One proposal here to mitigate risks would be to use some form of sandboxing where the Workflow executor is only able to run inside of an unprivileged Docker container. Such a solution would need to:

1. Mount the local working directory into the container so it is still editing the files the user is working on in the host.

1. Install all development dependencies the user or agent would